



UNIVERSITÀ
DEGLI STUDI
DI BERGAMO

Dipartimento
di Ingegneria Gestionale,
dell'Informazione e della Produzione

Logica Proposizionale

Intelligenza Artificiale

Ingegneria Informatica

Docente

Domenico Fabio Savo

La logica proposizionale

La **logica proposizionale**, come suggerisce il nome, è una logica che tratta di proposizioni (frasi) e della relazione tra loro.

Le affermazioni "***piove***" e "***la strada è bagnata***" sono proposizioni.

Queste proposizioni posso essere collegate tra loro per formare una nuova proposizione:
"se piove la strada è bagnata".

Scritto in modo più formale:

$piove \Rightarrow la\ strada\ è\ bagnata$.

Per lavorare in modo formale con la logica proposizionale è necessario definire in modo preciso come costruire le **formule della logica proposizionale** (sintassi).



Logica proposizionale: Sintassi

La **sintassi** della logica proposizionale definisce le formule accettabili.

Le **formule atomiche** consistono di un **simbolo proposizionale**

Es: A, B, C, ...

Due simboli proposizionali hanno un valore prefissato:

- **True** è la proposizione sempre vera,
- **False** quella sempre falsa.

Si possono costruire **formule complesse** partendo da formule più semplici e all'uso di parentesi e **connettivi logici**. Ci sono cinque connettivi di uso comune:

$\neg$ (not), $\wedge$ (and), $\vee$ (or), $\Rightarrow$ (implicazione), $\Leftrightarrow$ (se e solo se)



Connettivi logici

- $\neg$ (not):** una formula come $\neg A$ è chiamata la **negazione di A**. Si può usare il termine **letterale** (literal) per indicare una formula atomica (letterale positivo) o una formula atomica negata (letterale negativo).
- $\wedge$ (and):** una formula il cui connettivo principale è $\wedge$, come $B \wedge C$, prende il nome di **coniunzione**; le sue due parti si chiamano congiunti.
- $\vee$ (or):** una formula in cui il connettivo principale è $\vee$, come $(A \wedge B) \vee C$, è una **disgiunzione**; le sue parti sono dette disgiunti.
- $\Rightarrow$ (implicazione):** una formula come $(A \wedge B) \Rightarrow \neg C$ si chiama **implicazione**. La sua premessa è $(A \wedge B)$ e la sua conclusione è $\neg C$. Sono anche dette asserzioni **if-then**. A volte è anche indicata con $\rightarrow$.
- $\Leftrightarrow$ (se e solo se):** la formula $A \Leftrightarrow \neg B$ è una **equivalenza**, chiamata anche **bicondizionale**.



Grammatica formale della logica proposizionale

$formula \rightarrow formulaAtomica \mid formulaComplessa$

$formulaAtomica \rightarrow True \mid False \mid P \mid Q \mid R \mid \dots$

$formulaComplessa \rightarrow (formula) \mid [formula]$
 $\mid \neg formula$
 $\mid (formula \wedge formula)$
 $\mid (formula \vee formula)$
 $\mid (formula \Rightarrow formula)$
 $\mid (formula \Leftrightarrow formula)$

Precedenza operatori : $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$



Formule proposizionali: esempio

Le seguenti sono formule proposizionali (ben formate):

- a (atomo o formula atomica)
- $(a \wedge b)$
- $\neg((a \wedge b) \vee (c \wedge d))$

Le seguenti, invece, non lo sono:

- $a \wedge$
- $\vee b$
- $a \wedge b$ (mancano le parentesi, comunque a volte anche noi le omettiamo)



Logica proposizionale: Semantica

Con il termine **Semantica** si intende l'insieme di regole usate per determinare il **valore di verità** di una formula nei confronti di una particolare **Interpretazione**.

Nella logica proposizionale, una **Interpretazione** è una assegnazione che stabilisce il **valore di verità** (**true** o **false**) di ogni simbolo proposizionale.

Per esempio, se le formule nella KB fanno uso dei simboli proposizionali **A**, **B**, e **C**, una **interpretazione** possibile è: $I = \{A = \text{false}, B = \text{false}, C = \text{true}\}$

Con 3 simboli ci sono $2^3 = 8$ possibili **interpretazioni**



Semantica dei connettivi logici

Per calcolare il **valore di verità** (true o false) di qualsiasi formula della logica proposizionale rispetto ad una **interpretazione** definiamo, come prima cosa, la semantica (comportamento) dei connettivi logici.

Ad ogni connettivo logico associamo una funzione:

Unaria per $\neg$ **NOT**: $\{\text{true}, \text{false}\} \rightarrow \{\text{true}, \text{false}\}$

Binaria per $\wedge$ (**AND**), $\vee$ (**OR**), $\Rightarrow$ (**IMP**), $\Leftrightarrow$ (**SSE**): $\{\text{true}, \text{false}\} \times \{\text{true}, \text{false}\} \rightarrow \{\text{true}, \text{false}\}$

Intuitivamente, queste funzioni ci permettono di definire il significato di una formula.

Tipicamente, definiamo tali funzioni usando le **Tavole di Verità**



NOT: Tavola di verità

A	$\neg A$
True	False
False	True



AND: Tavola di verità

A	B	$A \wedge B$
True	True	True
True	False	False
False	True	False
False	False	False

OR: Tavola di verità

A	B	$A \vee B$
True	True	True
True	False	True
False	True	True
False	False	False



IMPLICAZIONE: Tavola di verità

A	B	$A \Rightarrow B$
True	True	True
True	False	False
False	True	True
False	False	True

Intuitivamente, l'asserzione $A \Rightarrow B$ significa: "se A è **vero**, allora sostengo che lo è anche B . In caso contrario non faccio alcuna asserzione"



SE e SOLO SE (SSE): Tavola di verità

A	B	$A \Leftrightarrow B$
True	True	True
True	False	False
False	True	False
False	False	True

Da qui in avanti abbrevieremo il nome del connettivo **Se e Solo Se** con **SSE**



Interpretazioni proposizionali

La semantica deve specificare come calcolare il valore di verità di una formula rispetto ad una **interpretazione**.

Sia $\mathcal{L}$ l'insieme delle formule proposizionali.

Una **interpretazione** è una funzione $I : \mathcal{L} \rightarrow \{true, false\}$ tale che:

- $I(\text{True}) = true$
- $I(\text{False}) = false$
- $I(a) \in \{true, false\}$, per ogni atomo $a \in \mathcal{L}$
- $I(\neg f) = \text{NOT}(I(f))$
- $I(f \wedge f') = \text{AND}(I(f), I(f'))$
- $I(f \vee f') = \text{OR}(I(f), I(f'))$
- $I(f \Rightarrow f') = \text{IMP}(I(f), I(f'))$
- $I(f \Leftrightarrow f') = \text{SSE}(I(f), I(f'))$

f e f' indicano delle
formule proposizionali
(anche atomiche)



Valutazione di una formula proposizionale

Consideriamo la formula $(A \wedge B) \vee ((\neg B) \wedge C)$ e l'interpretazione I tale che:
 $I(A) = \text{true}$, $I(B) = \text{false}$, $I(C) = \text{true}$

Abbiamo che:

- $AND(I(A), I(B)) = \text{false}$
- $NOT(I(B)) = \text{true}$
- $AND(\text{true}, I(C)) = \text{true}$
- $OR(\text{false}, \text{true}) = \text{true}$

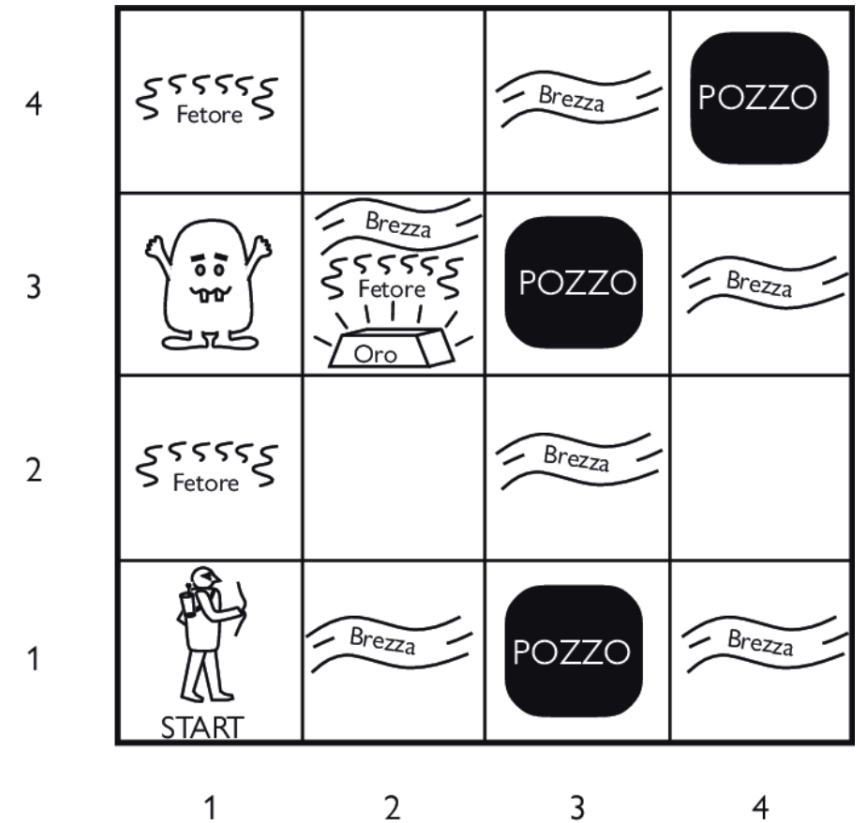


Il mondo del Wumpus

Come esempio di come può operare un **agente basato sulla logica** usiamo il mondo del Wumpus. Il cui ambiente è descritto come segue:

Ambiente:

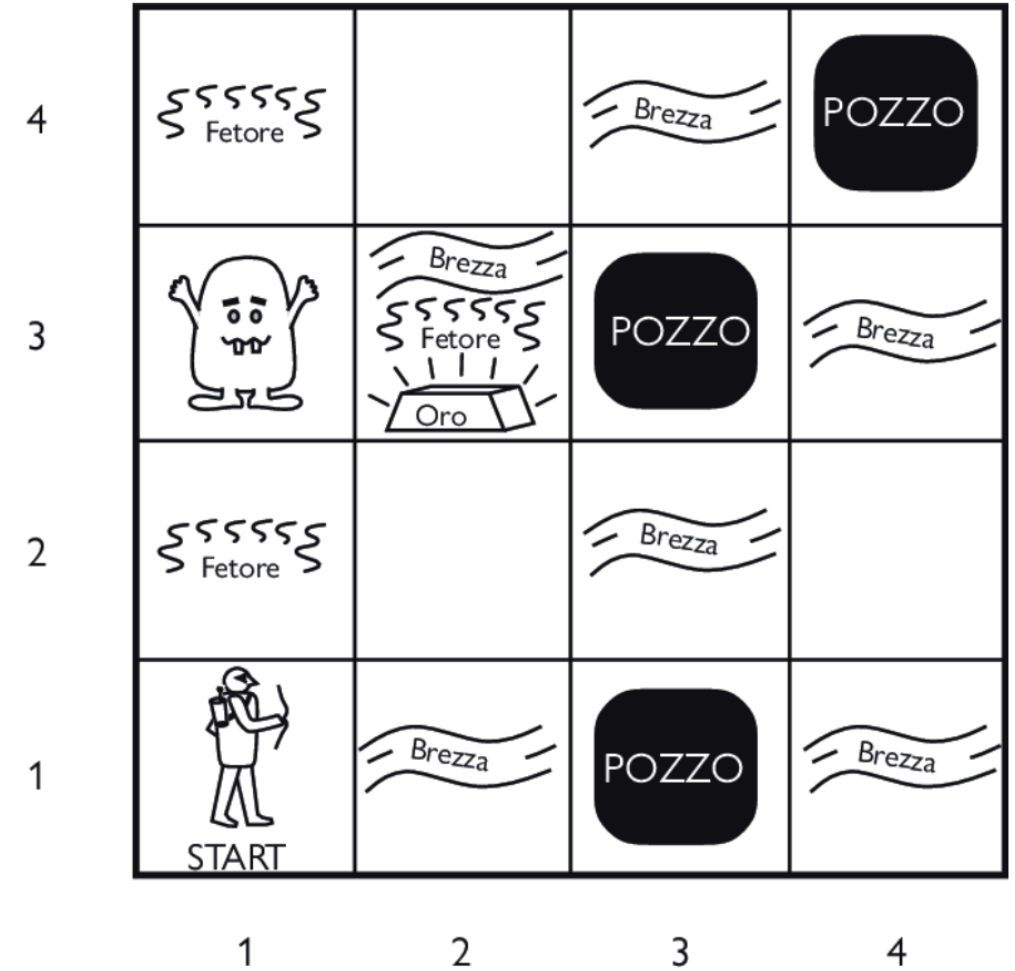
- Una caverna formata 4x4 stanze collegate da passaggi.
- In una di queste stanze c'è il mostro Wumpus.
- Il Wumpus può essere colpito da lontano da un agente, che però ha una sola freccia.
- Alcune stanze contengono pozzi senza fondo che intrappoleranno chiunque entri (tranne il Wumpus).
- In una delle stanze c'è un tesoro.
- L'agente comincia sempre nella posizione etichettata [1,1] rivolto a est.



Il mondo del Wumpus

Sensori: l'agente ha cinque sensori, ognuno dei quali fornisce un singolo bit di informazione:

- Nei riquadri direttamente adiacenti (non in diagonale) a quello che contiene il Wumpus, l'agente percepirà un **Fetore**;
- Nei riquadri direttamente adiacenti a un pozzo l'agente percepirà una **Brezza**;
- Nel riquadro che contiene l'oro l'agente percepirà uno **Scintillio**;
- Qualora l'agente dovesse sbattere contro un muro, percepirà un **Urto**;
- Quando il Wumpus viene ucciso emette un **Ululato** percepibile nell'intera caverna.



Il mondo del Wumpus

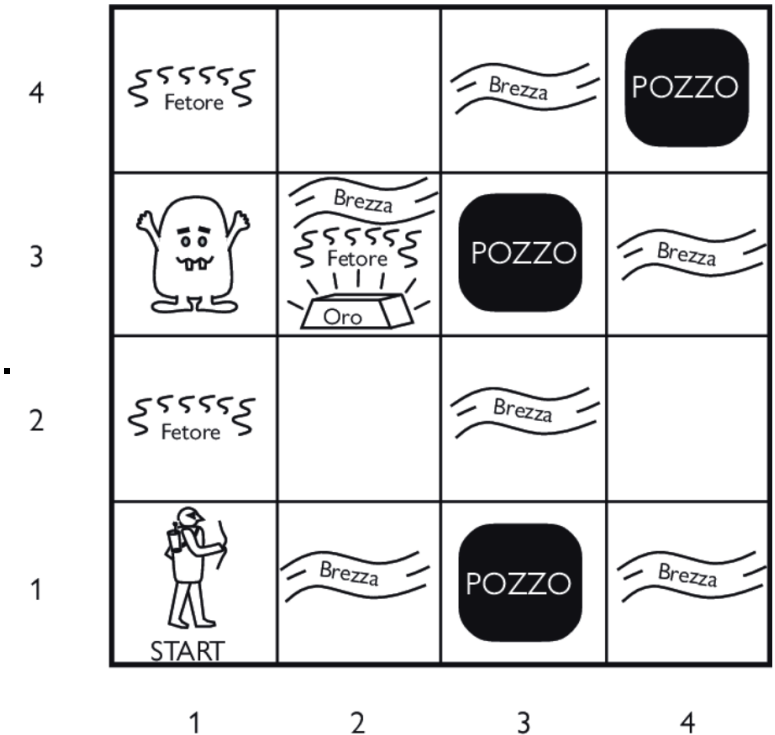
Misura di prestazione:

- +1000 se si esce dalla caverna con l'oro,
- -1000 se si muore (in un pozzo o per il Wumpus),
- -1 per ogni azione eseguita
- -10 per l'uso della freccia.

Il gioco termina quando si muore o quando si esce dalla caverna.

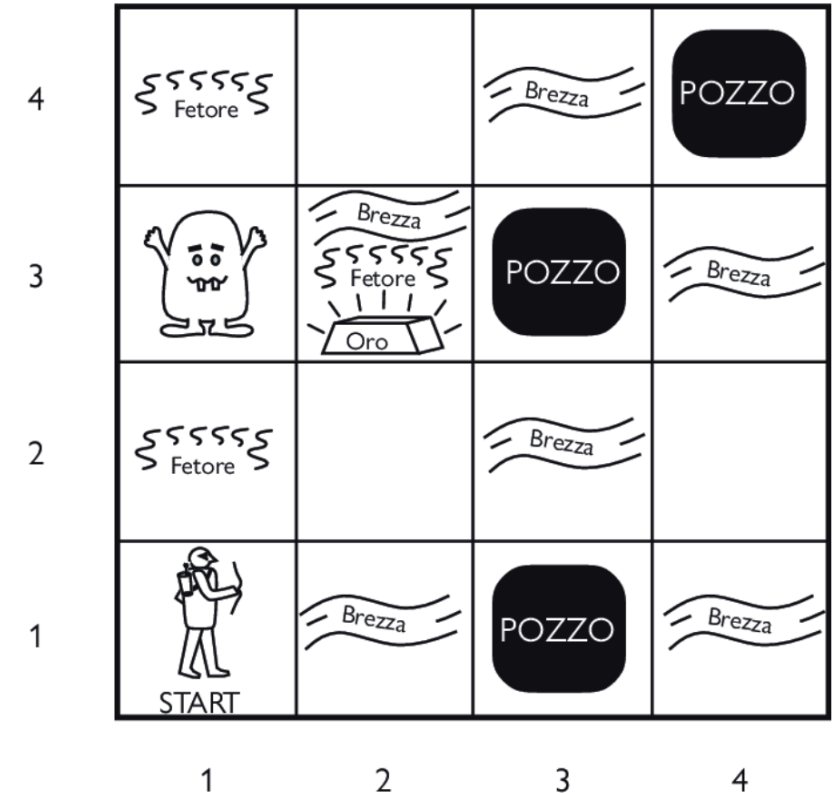
Attuatori:

- Muoversi in **Avanti** (se c'è un muro non si muove).
- **GiraSinistra** di 90°
- **GiraDestra** di 90°.
- **Afferra** (prendere l'oro se è nella stanza)
- **Scocca** feccia in linea retta nella direzione in cui è rivolto (la freccia avanza finché non colpisce il Wumpus, uccidendolo, o un muro). Può essere usata solo una volta.
- **Esci** per arrampicarsi fuori dalla caverna, ma solo dalla stanza [1,1].



Il mondo del Wumpus: caratterizzazione

- Osservabile?
No - solo percezione locale
- Deterministico?
Sì - i risultati sono esattamente specificati
- Episodico?
No - sequenziale a livello di azioni
- Statico?
Sì – pozzi e Wumpus e stanze non si muovono.
- Discreto?
Sì
- Singolo agente?
Sì – il Wumpus è essenzialmente una caratteristica naturale

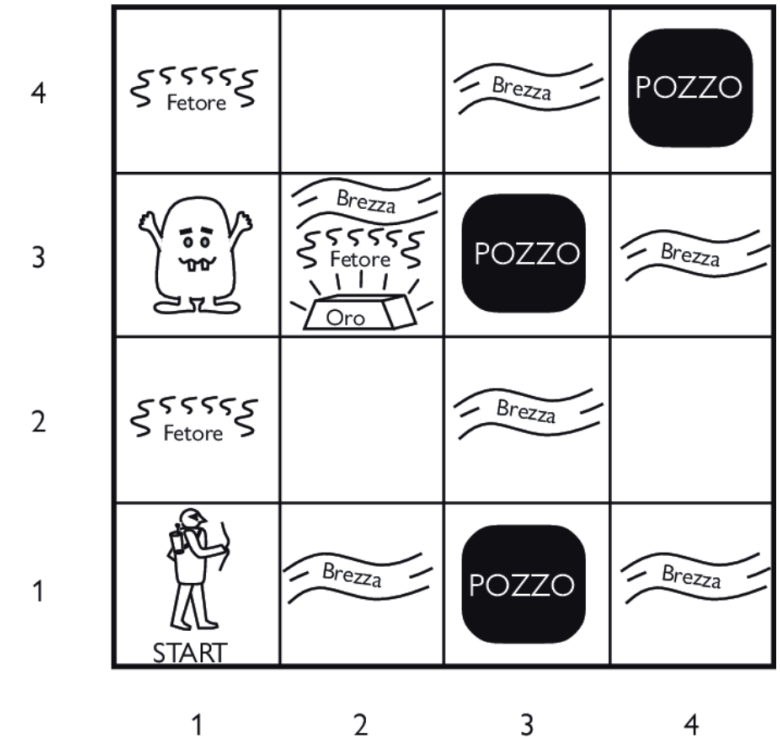


Il mondo del Wumpus come base di conoscenza

Definiamo una base di conoscenza per il mondo del Wumpus.

Utilizzeremo la seguente notazione:

- Ogni riquadro è indicato da due coordinate **[colonna,riga]**.
- $P_{x,y}$ è vero se esiste un pozzo in $[x, y]$.
- $W_{x,y}$ è vero se esiste un wumpus in $[x, y]$, morto o vivo.
- $B_{x,y}$ è vero se esiste una brezza in $[x, y]$.
- $S_{x,y}$ è vero se esiste un fetore in $[x, y]$.
- $L_{x,y}$ è vero se l'agente si trova nella locazione $[x, y]$.
- Ogni formula ha una etichetta del tipo R_i



Il mondo del Wumpus come base di conoscenza

Iniziamo descrivendo gli aspetti immutabili del ambiente. Cioè indichiamo quelle formule vere in qualunque configurazione del ambiente.

- In $[1,1]$ non ci sono pozzi:
 $R_1 : \neg P_{1,1}$
- In una stanza si percepisce brezza se e solo se c'è un pozzo in una stanza adiacente. Questo dev'essere specificato per ogni locazione; per ora indichiamo solo quelle rilevanti:

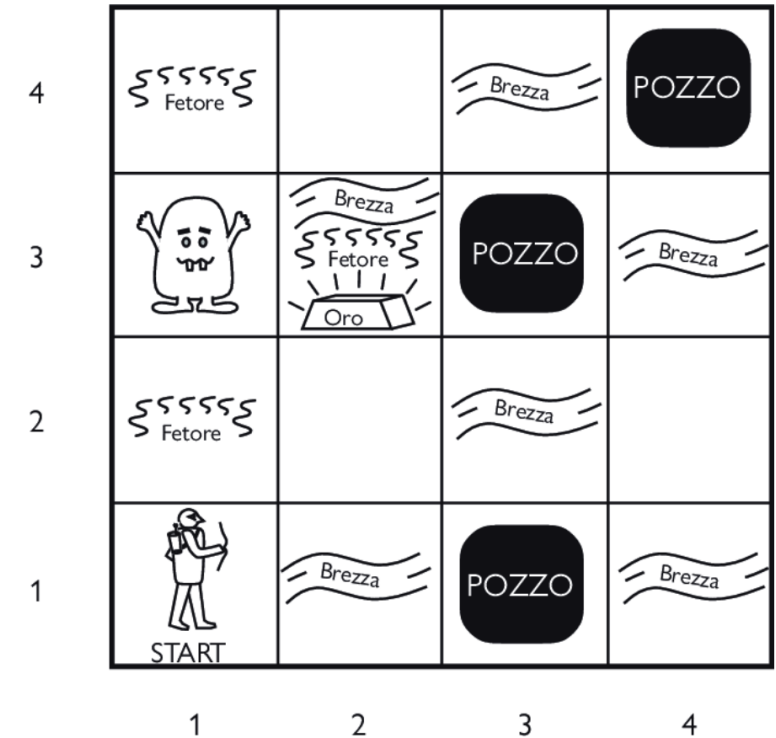
$$R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

$$R_3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$$

- Aggiungiamo le percezioni di brezza raccolte nelle prime due stanze visitate dall'agente nella configurazione in figura:

$$R_4 : \neg B_{1,1}$$

$$R_5 : B_{2,1}$$



Una semplice procedura di inferenza

Vogliamo determinare se la stanza [1,2] non contiene un pozzo. Per esserne sicuri dobbiamo quindi verificare se $\neg P_{1,2}$ è **conseguenza logica** della base di conoscenza **K** del mondo del Wumpus, cioè se $K \models \neg P_{1,2}$

Un approccio basato sul **model checking** costruirà tutte le possibili interpretazioni in modo da individuare i modelli della base di conoscenza (KB) e quindi verificherà se questi sono modelli anche per $\neg P_{1,2}$

Considerando solo i **7** simboli $B_{1,1}$, $B_{2,1}$, $P_{1,1}$, $P_{1,2}$, $P_{2,1}$, $P_{2,2}$ e $P_{3,1}$ avremo **$2^7 = 128$** possibili interpretazioni. Solo tre di essi sono modelli per K. In tutti e tre i modelli $P_{1,2}$ è **falsa** (quindi $\neg P_{1,2}$ è vera) e quindi possiamo essere certi che non c'è pozzo in [1,2].

Abbiamo dimostrato che $K \models \neg P_{1,2}$

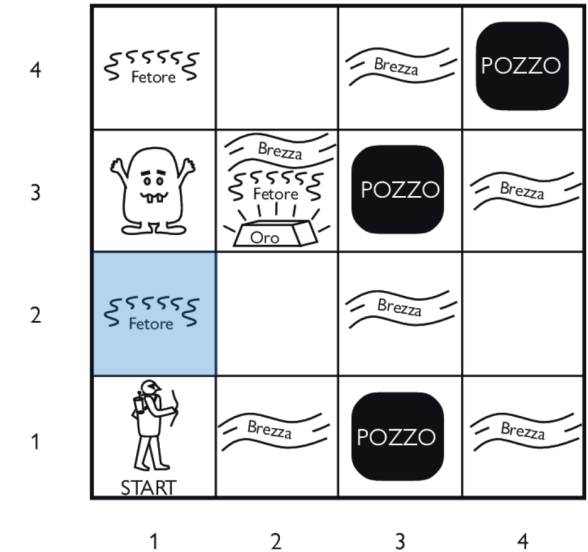


Tavola di verità della base di conoscenza

Interpretazioni	$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	K	$\neg P_{1,2}$	modelli
	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	
	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>true</i>	
	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	
	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	
	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<u><i>true</i></u>	<u><i>true</i></u>	
	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<u><i>true</i></u>	<u><i>true</i></u>	
	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>	<u><i>true</i></u>	<u><i>true</i></u>	
	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	
	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	
	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>false</i>	

Verificare $K \models \alpha$ con il model checking

Un approccio basato sul **model checking** per verificare se $K \models \alpha$ richiede di:

1. Costruire tutte le possibili interpretazioni dei simboli proposizionali di K ed α
2. Costruire l'insieme $M(K)$ determinando quali tra tali interpretazioni è un modello per K
3. Verificare se ogni interpretazione in $M(K)$ è un modello per α

Un algoritmo così definito è **corretto** (infatti implementa direttamente la definizione di conseguenza logica), e **completo**, perché funziona per ogni K e α . Inoltre termina sempre dato che il numero delle interpretazioni da esaminare è finito.

Complessità: Se il numero totale di simboli contenuti in K e α è n , il numero di interpretazioni è 2^n . La **complessità temporale** dell'algoritmo è quindi **$O(2^n)$** .

Risultato non inaspettato dato che è dimostrato che la **conseguenza logica proposizionale** è **co-NP-completa**, perciò ogni algoritmo di inferenza conosciuto per la logica proposizionale ha una complessità temporale, nel caso peggiore, **esponenziale nelle dimensioni dell'input**.



Verificare $K \models \alpha$ con il model checking: complessità spaziale

Il semplice algoritmo di inferenza presentato costruisce tutte le interpretazioni al fine di verificare se tutti i modelli per K sono anche modelli per α .

La **complessità temporale** e **spaziale** di tale algoritmo coincidono e sono pari a **$O(2^n)$** .

Esistono algoritmi per verificare se $K \models \alpha$ attraverso il model checking più efficienti rispetto alla complessità spaziale.

In particolare, si può definire un algoritmo basato sulla **ricerca in profondità** che non ha bisogno di memorizzare tutte le interpretazioni ed è caratterizzato da una complessità spaziale di **$O(n)$** .



La dimostrazione di teoremi

Un differente approccio per determinare la conseguenza logica è quello basato sulla **dimostrazione di teoremi**: per dimostrare che $\mathbf{K} \models \alpha$ applichiamo delle **regole di inferenza** direttamente alle formule della base di conoscenza $\mathbf{K}$ al fine di costruire una dimostrazione della formula α (senza calcolare i modelli di $\mathbf{K}$ e α).

Le **regole di inferenza** sono spesso scritte come:

$$\frac{A_1 \dots A_n}{A}$$

$$\frac{\text{premesse}}{\text{conclusioni}}$$

Useremo inoltre i seguenti concetti relativi alla conseguenza logica:

- **Equivalenza logica**
- **Validità di una formula**
- **Soddisfacibilità di una formula**



Equivalenza logica

Due formule α e β sono **logicamente equivalenti** (denotato con $\alpha \equiv \beta$) se hanno gli stessi modelli, cioè sono vere nello stesso insieme di interpretazioni.

Avremo quindi che due formule α e β sono **logicamente equivalenti** se e soltanto se ognuna di esse è conseguenza logica dell'altra:

$$\alpha \equiv \beta \text{ se e solo se } \alpha \models \beta \text{ e } \beta \models \alpha$$

È facile, ad esempio, dimostrare con le tavole di verità la seguente equivalenza:

$$P \wedge Q \equiv Q \wedge P$$



Equivalenze logiche notevoli

$(\alpha \wedge \beta)$	$\equiv$	$(\beta \wedge \alpha)$	commutatività di $\wedge$
$(\alpha \vee \beta)$	$\equiv$	$(\beta \vee \alpha)$	commutatività di $\vee$
$((\alpha \wedge \beta) \wedge \gamma)$	$\equiv$	$(\alpha \wedge (\beta \wedge \gamma))$	associatività di $\wedge$
$((\alpha \vee \beta) \vee \gamma)$	$\equiv$	$(\alpha \vee (\beta \vee \gamma))$	associatività di $\vee$
$\neg(\neg\alpha)$	$\equiv$	α	eliminazione della doppia negazione
$(\alpha \Rightarrow \beta)$	$\equiv$	$(\neg\beta \Rightarrow \neg\alpha)$	contrapposizione
$(\alpha \Rightarrow \beta)$	$\equiv$	$(\neg\alpha \vee \beta)$	eliminazione dell'implicazione
$(\alpha \Leftrightarrow \beta)$	$\equiv$	$(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$	eliminazione del bicondizionale
$\neg(\alpha \wedge \beta)$	$\equiv$	$(\neg\alpha \vee \neg\beta)$	De Morgan
$\neg(\alpha \vee \beta)$	$\equiv$	$(\neg\alpha \wedge \neg\beta)$	De Morgan
$(\alpha \wedge (\beta \vee \gamma))$	$\equiv$	$(\alpha \wedge \beta) \vee (\alpha \wedge \gamma)$	distributività di $\wedge$ su $\vee$
$(\alpha \vee (\beta \wedge \gamma))$	$\equiv$	$(\alpha \vee \beta) \wedge (\alpha \vee \gamma)$	distributività di $\vee$ su $\wedge$



Equivalenze logiche: esercizio

Dimostrare utilizzando le tavole di verità (model checking) le seguenti equivalenze logiche:

- $(A \Rightarrow B) \equiv (\neg A \vee B)$
- $\neg(A \wedge B) \equiv (\neg A \vee \neg B)$



Validità di una formula

Una formula α è **valida** se è vera in tutte le interpretazioni (tutte le interpretazioni sono suoi modelli)

- Le formule valide sono note anche come **tautologie**.
- La formula **True** è per definizione vera in ogni interpretazione, quindi ogni formula **valida** è logicamente equivalente a **True**.

Esempi di formule valide sono:

- $P \wedge \neg P$
- $A \Rightarrow A$
- $(A \wedge (A \Rightarrow B)) \Rightarrow B$



Teorema di deduzione

La nozione di validità insieme a quella di equivalenza logica permettono di fornire il seguente **teorema di deduzione**

Date due formule α e β , abbiamo che $\alpha \models \beta$ se e solo se la formula $(\alpha \Rightarrow \beta)$ è **valida**.

Il teorema di deduzione ci dice che possiamo decidere se $\alpha \models \beta$ verificando che $(\alpha \Rightarrow \beta)$ sia vera in ogni interpretazione, oppure dimostrando che $(\alpha \Rightarrow \beta)$ è equivalente a **True**.

Viceversa, il teorema di deduzione afferma che ogni formula di **implicazione** valida descrive un'**inferenza legittima**.



Soddisfacibilità di una formula

Una formula α è **soddisfacibile** se è vera in almeno una interpretazione (esiste almeno una interpretazione che è modello per α).

Se non esiste alcun modello per una formula allora questa si dice **insoddisfacibile**.

La **soddisfacibilità** di una formula può essere dimostrata enumerando tutte le possibili interpretazioni finché non se ne troverà una in cui la formula è soddisfatta (è vera).

Il problema di determinare la soddisfacibilità delle formule della logica proposizionale viene chiamato problema SAT ed è stato il primo problema di cui è stata dimostrata la **NP-completezza**.



Soddisfacibilità e validità

Validità e soddisfacibilità sono strettamente connesse:

Una formula α è **valida** se e solo se $\neg\alpha$ è **insoddisfacibile**; di contro, α è **soddisfacibile** se e solo se $\neg\alpha$ **non è valida**

Da tali proprietà e dal **teorema di deduzione** segue il seguente teorema:

Date due formule α e β , si ha che $\alpha \models \beta$ se e solo se la formula $\neg(\alpha \Rightarrow \beta)$ è **insoddisfacibile**.

Dimostrare β partendo da α verificando l'insoddisfacibilità di $(\alpha \wedge \neg\beta)$ corrisponde alla tecnica matematica di dimostrazione chiamata **dimostrazione per assurdo**: Si assume che la formula β sia **falsa** e si dimostra che questo porta a una **contraddizione** con gli assiomi α (che assumiamo veri).



Dimostrazione per assurdo: esercizio

Esercizio: Fornire una dimostrazione per assurdo del **teorema di deduzione**:

Date due formule α e β , abbiamo che $\alpha \models \beta$ se e solo se la formula $(\alpha \Rightarrow \beta)$ è **valida**.



Regole di inferenza per la logica proposizionale

Vedremo ora due regole di inferenza utili per la dimostrazione di teoremi.

Modus Ponens (dal latino, “modo che afferma”):

$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta}$$

AND Elimination:

$$\frac{\alpha_1 \wedge \alpha_2 \wedge \dots \wedge \alpha_n}{\alpha_i}$$

La notazione significa che, ogni volta che sono date per vere le *premesse*, ad esempio le formule $\alpha \Rightarrow \beta$ e α , allora si possono inferire le *conclusioni*, cioè la formula β



Altre regole di inferenza

Tutte le **equivalenze logiche notevoli** posso essere usate come regole di inferenza.

Ad esempio **l'equivalenza per l'eliminazione dell'implicazione** dà origine alle seguenti due regole di inferenza:

$$\frac{\alpha \Rightarrow \beta}{\neg \alpha \vee \beta}$$

$$\frac{\neg \alpha \vee \beta}{\alpha \Rightarrow \beta}$$



Applicazione delle regole di inferenza: esempio

Sappiamo che i **gatti** sono **felini** e che i **felini** sono **animali**, sappiamo inoltre di avere un **gatto**. **Possiamo dedurre di avere un animale?**

La base di conoscenza che modella quanto detto è la seguente:

$K = \{ \text{gatto} \Rightarrow \text{felino}, \text{felino} \Rightarrow \text{animale}, \text{gatto} \}$

Applicando la modul ponens (MP) sulle formule $\text{gatto} \Rightarrow \text{felino}$ e gatto deduciamo di avere un **felino**, cioè $K \vdash_{\text{MP}} \text{felino}$

$$\frac{\text{gatto} \Rightarrow \text{felino}, \text{gatto}}{\text{felino}}$$

Applicando ancora la MP sulle formule **felino** (derivata in precedenza) e $\text{felino} \Rightarrow \text{animale}$ deduciamo di avere un **animale**, cioè $K \vdash_{\text{MP}} \text{animale}$ la risposta alla domanda è quindi **si**

$$\frac{\text{felino} \Rightarrow \text{animale}, \text{felino}}{\text{animale}}$$



Inferenza nel mondo del Wumpus: esempio (1\2)

Vediamo come utilizzare le regole di inferenza viste nel mondo del Wumpus.

Ricordiamo che la base di conoscenza del Wumpus contiene le seguenti formule:

- $R_1 : \neg P_{1,1}$ (in $[1,1]$ non ci sono pozzi)
 - $R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
 - $R_3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$
 - $R_4 : \neg B_{1,1}$ (percezione: non c'è brezza in $[1,1]$)
 - $R_5 : B_{2,1}$ (percezione: c'è brezza in $[2,1]$)
- } (In una stanza si percepisce brezza se e solo se c'è un pozzo in una stanza adiacente)

Vogliamo dimostrare $\neg P_{1,2}$ ($K \models \neg P_{1,2}$), cioè che in $[1,2]$ non ci sono pozzi.



Inferenza nel mondo del Wumpus: esempio (2\2)

1. Applichiamo l'eliminazione del bicondizionale a R_2 (cioè $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$) per ottenere:
 $R_6 : (B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$
2. Applichiamo l'eliminazione degli and a R_6 per ottenere:
 $R_7 : ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$
3. L'equivalenza logica della contrapposizione fornisce:
 $R_8 : (\neg B_{1,1} \Rightarrow \neg (P_{1,2} \vee P_{2,1}))$
4. Applichiamo Modus Ponens con R_8 e la percezione R_4 (cioè $\neg B_{1,1}$), ottenendo:
 $R_9 : \neg (P_{1,2} \vee P_{2,1})$
5. Applichiamo la regola di De Morgan, arrivando alla conclusione:
 $R_{10} : \neg P_{1,2} \wedge \neg P_{2,1}$
Ovvero, non ci sono pozzi né in [1,2] né in [2,1].

Inferenza nel mondo del Wumpus: osservazioni

La ricerca di dimostrazioni, quindi, è un'alternativa all'enumerazione di modelli.

In molti casi pratici, trovare una dimostrazione può essere molto efficiente semplicemente perché si possono ignorare le proposizioni irrilevanti, indipendentemente dal loro numero.

Per esempio, la dimostrazione appena fornita per $\neg P_{1,2} \wedge \neg P_{2,1}$ non fa alcuna menzione delle proposizioni $B_{2,1}$, $P_{1,1}$, $P_{2,2}$ o $P_{3,1}$, che possono essere ignorate. Infatti la proposizione obiettivo $P_{1,2}$ compare solo nella formula R_2 , e le altre proposizioni in R_2 compaiono solo in R_4 e R_2 ; ne consegue che R_1 , R_3 e R_5 non hanno alcun ruolo nella dimostrazione.

Lo stesso ragionamento vale anche se dovessimo aggiungere un **milione** di altre formule alla base di conoscenza: **il semplice algoritmo della tavola di verità, al contrario, sarebbe presto sopraffatto dalla crescita esponenziale del numero di modelli.**



Monotonicità

Una importante proprietà della logica proposizionale è quella della **monotonicità** che afferma che l'insieme di formule che sono conseguenze logiche di una base di conoscenza **K** **può solo aumentare** all'aumentare delle informazioni aggiunte a **K**

Formalmente:

Per qualsiasi coppia di formule α e β e base di conoscenza **K**, se $\mathbf{K} \models \alpha$ allora $\mathbf{K} \cup \{\beta\} \models \alpha$

La **monotonicità** significa che **le regole di inferenza** possono essere applicate non appena si trovano nella base di conoscenza le **premesse** necessarie; le **conclusioni** di tali regole dovranno essere vere **indipendentemente dal resto delle formule contenute nella base di conoscenza**.



Problema di Ricerca di Dimostrazioni

Nell'esempio del Wumpus abbiamo visto come le regole di inferenza possono essere utilizzate per dimostrare le conseguenze logiche (il teorema).

Come facciamo a scegliere quali regole applicare e in quale ordine?

In realtà, dato un insieme di regole di inferenza, potremmo applicare uno degli algoritmi di ricerca visti per trovare una sequenza di passi che costituisca una dimostrazione.

Supponiamo di voler dimostrare che $K \models \alpha$, possiamo definire il problema di ricerca corrispondente (**ricerca di dimostrazioni**) nel modo seguente:

- **Stato iniziale:** la base di conoscenza K iniziale.
- **Azioni:** l'insieme delle azioni consiste di tutte le regole di inferenza applicate a tutte le formule di K che corrispondono ("fanno match") alle *premesse* della regola di inferenza.
- **Risultato:** il risultato di un'azione è l'aggiunta a K delle *conclusioni* della regola di inferenza.
- **Obiettivo:** l'obiettivo è uno stato che contiene la formula α (da dimostrare).



Dimostrazione per Risoluzione

Le **regole di inferenza** viste fin qui sono **corrette**, ma cosa sappiamo della **completezza**?

Anche applicando al **problema della ricerca di dimostrazione** un algoritmo di ricerca che sappiamo essere **completo** (come ad esempio la *ricerca per approfondimento iterativo*) ma utilizzando un insieme di regole di inferenza non adeguato (completo), l'algoritmo non riuscirà a trovare un percorso per raggiungere lo stato obiettivo (non riuscirà quindi a completare la dimostrazione).

Introdurremo ora una singola regola di inferenza, chiamata **principio di risoluzione**, che se *usata da qualsiasi algoritmo di ricerca completo* dà luogo a un **algoritmo di inferenza completo**.

La regola di inferenza **principio di risoluzione** si applica solo a **clausole**, cioè una formula costituita dalla sola **disgiunzione** (or) di **letterali** (cioè formule atomiche negate e non negate).



Principio di risoluzione

Siano $C_1 = l_1 \vee \dots \vee l_k$ e $C_2 = m_1 \vee \dots \vee m_n$ due clausole tale per cui esistono due letterali l_i in C_1 e m_t in C_2 che sono tra loro **complementari** (cioè, uno è la negazione dell'altro, es: P e $\neg P$), avremo quindi:

Principio di risoluzione:

$$\frac{l_1 \vee \dots \vee l_k, m_1 \vee \dots \vee m_n}{l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_k \vee m_1 \vee \dots \vee m_{t-1} \vee m_{t+1} \vee \dots \vee m_n}$$

Nota: un singolo letterale può essere considerato come una disgiunzione formata da un solo letterale (**clausola unitaria**).

Un dimostratore di teoremi basato sulla risoluzione può, per ogni coppia di formule α e β della logica proposizionale, decidere se $\alpha \models \beta$.



Principio di risoluzione: correttezza

Per dimostrare la correttezza del principio di risoluzione consideriamo il caso di due clausole formate da solo due letterali (la dimostrazione può poi essere estesa al caso generale).

Siano $C_1 = \alpha \vee \beta$ e $C_2 = \gamma \vee \neg\beta$ (con α, β, γ formule atomiche). Secondo il **principio di risoluzione** avremo:

$$\frac{\alpha \vee \beta, \gamma \vee \neg\beta}{\alpha \vee \gamma}$$

Dimostrazione: Sappiamo che se le *premesse* sono *vere*, allora lo saranno anche le *conclusioni*. Nel nostro caso, questo significa che ogni interpretazione che è modello sia per C_1 che per C_2 sarà anche modello per $\alpha \vee \gamma$. Sia I tale interpretazione. Si possono distinguere due casi: (i) β è *vero* in I oppure (ii) β è *falso* in I . Nel primo caso, affinché C_2 sia *vera* in I dobbiamo avere che γ è *vera* in I (poiché $\neg\beta$ è *falso*). Quindi $\alpha \vee \gamma$ è *vero* in I . Nel secondo caso affinché C_1 sia *vera* in I dobbiamo avere che α è *vera* in I (poiché β è *falso*). Quindi $\alpha \vee \gamma$ è *vero* in I . In entrambi i casi quindi $\alpha \vee \gamma$ è *vero* in I .

Forma normale congiuntiva

La regola di risoluzione si applica solo a clausole (disgiunzioni di letterali), questo però non la rende meno generale poiché **ogni formula della logica proposizionale è logicamente equivalente a una congiunzione di clausole**.

Una formula così espressa viene detta in **forma normale congiuntiva** o, con l'acronimo inglese, **CNF**

Un formula è in **CNF** se è del tipo:

$$\gamma_1 \wedge \gamma_2 \wedge \cdots \wedge \gamma_k$$

Dove ogni γ_i è una **clausola**, ovvero una formula del tipo:

$$l_1 \vee l_2 \vee \cdots \vee l_n$$

Dove ogni l_i è un **letterale**, cioè una variabile proposizionale o la **negazione** di una variabile proposizionale (es: P o $\neg Q$)



Forma normale congiuntiva: esempio

Trasformiamo la formula proposizionale $A \Leftrightarrow (B \vee C)$ in una formula logicamente equivalente in CNF

- Applico l'eliminazione del bicondizionale:

$$(A \Rightarrow (B \vee C)) \wedge ((B \vee C) \Rightarrow A)$$

- Applico l'eliminazione dell'implicazione:

$$(\neg A \vee B \vee C) \wedge (\neg(B \vee C) \vee A)$$

- Applico De Morgan a $\neg(B \vee C)$

$$(\neg A \vee B \vee C) \wedge (\neg B \wedge \neg C) \vee A$$

- Applico la distribuzione dell' $\vee$:

$$(\neg A \vee B \vee C) \wedge (\neg B \vee A) \wedge (\neg C \vee A)$$

La formula è in **CNF**



Normalizzazione di una formula in CNF

In generale la procedura per trasformare una qualunque formula proposizionale in una formula equivalente in CNF applichiamo la seguente procedura:

1. Eliminiamo tutti gli operatori bicondizionali $\Leftrightarrow$
2. Eliminiamo tutti i simboli di implicazione $\Rightarrow$
3. Si utilizzano ripetutamente le leggi di **De Morgan** e la legge **della doppia negazione** per portare le negazioni davanti alle variabili proposizionali
4. Applichiamo le proprietà distributive dell'**AND** e dell'**OR** fino ad ottenere un CNF



Algoritmo di Risoluzione

Le procedure di inferenza basate sulla risoluzione sfruttano il principio di dimostrazione per assurdo: per dimostrare che $K \models \alpha$, dimostriamo che $(K \wedge \neg\alpha)$ è **insoddisfacibile**. Dobbiamo quindi giungere a una **contraddizione**.

L'algoritmo di risoluzione per prima cosa si converte $(K \wedge \neg\alpha)$ in CNF e si applica ripetutamente la regola di risoluzione alle clausole risultanti:

- Ogni coppia che contiene letterali complementari è risolta per produrre una nuova clausola che viene aggiunta all'insieme delle formule (se non è già presente).

Il processo continua finché non si verifica una delle due seguenti possibilità:

- non è più possibile aggiungere alcuna clausola, nel qual caso α non è conseguenza logica di K ;
- la risoluzione applicata a due clausole dà come risultato la **clausola vuota**, nel qual caso da K consegue logicamente α .

La **clausola vuota** può avere origine solo dalla risoluzione di due **clausole unitarie** complementari come, ad esempio, P e $\neg P$.



Algoritmo di Risoluzione: pseudocodice

```
function PL-Risoluzione( $K, \alpha$ ) returns true oppure false  
  inputs: una base di conoscenza  $K$ , una formula proposizionale  $\alpha$   
   $clausole \leftarrow$  l'insieme di clausole nella rappresentazione CNF di  $(K \wedge \neg \alpha)$   
   $nuove \leftarrow \{ \}$   
  loop do  
    for each coppia di clausole  $C_i, C_j$  in  $clausole$  do  
       $risolventi \leftarrow$  PL-Risolvi( $C_i, C_j$ )  
      if  $risolventi$  contiene la clausola vuota then return true  
       $nuove \leftarrow nuove \cup risolventi$   
    if  $nuove \subseteq clausole$  then return false  
     $clausole \leftarrow clausole \cup nuove$ 
```

La funzione **PL-Risolvi** restituisce l'insieme di tutte le possibili clausole ottenute risolvendo i due input.



Algoritmo di Risoluzione: esempio del Wumpus (1\2)

Applichiamo l'algoritmo di risoluzione al caso del Wumpus.

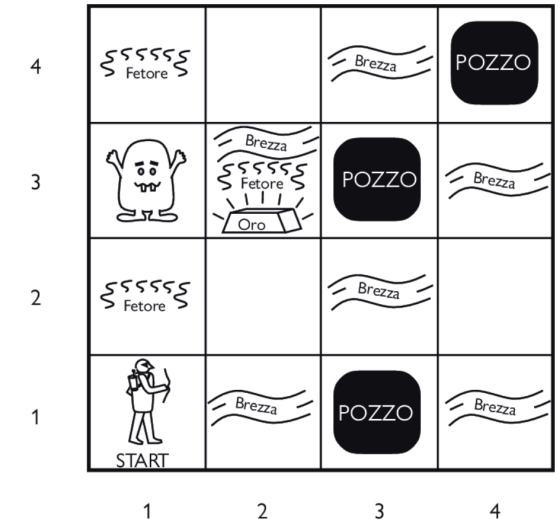
Quando l'agente si trova in [1,1] non percepisce alcun spostamento d'aria, per cui non ci possono essere pozzi nelle stanze adiacenti.
La base di conoscenza relativa è:

$$K = R_2 \wedge R_4 = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1}$$

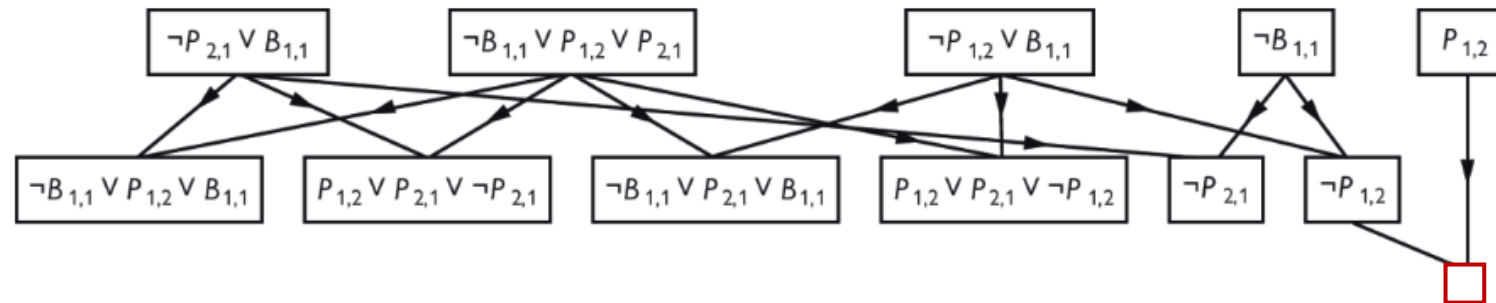
Vogliamo dimostrare $\neg P_{1,2}$ segue logicamente da K (i.e., $K \models \neg P_{1,2}$).

Convertendo $(K \wedge \neg \alpha)$ in CNF otteniamo:

$$(\neg B_{1,1} \vee P_{2,1} \vee P_{1,2}) \wedge (B_{1,1} \vee \neg P_{2,1}) \wedge (B_{1,1} \vee \neg P_{1,2}) \wedge (\neg B_{1,1}) \wedge (P_{1,2})$$



Algoritmo di Risoluzione: esempio del Wumpus (2\2)



La figura mostra una parziale applicazione dell'algoritmo in cui i vari congiunti della formula iniziale (prima riga) vengono tra loro accoppiati (in ogni modo) nel tentativo di applicare il principio di risoluzione che produce le formule nella seconda riga. Si può vedere che ad un certo punto l'algoritmo produrrà il seguente cammino di risoluzione che porta alla **clausola vuota** (false)

$$\frac{P_{1,2} \quad \frac{\neg B_{1,1} \quad \neg P_{1,2} \vee B_{1,1}}{\neg P_{1,2}}}{\{}}$$

Clausole di Horn e clausole definite

Alcune basi di conoscenza del mondo reale soddisfano certe restrizioni sulla forma delle formule che contengono, tali forme ristrette sono dette **Clausole di Horn**.

Su basi di conoscenza contenenti solo questo tipo di clausole è possibile utilizzare un algoritmo di inferenza più ristretto ed **efficiente** di quello di **risoluzione**.

Una clausola di Horn può essere di due tipi:

Clausola definita: Una disgiunzione di letterali in cui **al massimo** un letterale è positivo

Es: $(A \vee \neg B \vee \neg C)$ oppure (A)

Clausola obiettivo: Una disgiunzione di letterali in cui **non compare** alcun letterale positivo

Es: $(\neg A \vee \neg B \vee \neg C)$



Clausole di Horn e clausole definite

Le basi di conoscenza contenenti soltanto **clausole definite** sono interessanti per le seguenti ragioni:

- ❑ Ogni **clausola definita** può essere scritta come un'**implicazione** la cui **premessa** è una congiunzione di letterali positivi e la cui **conclusione** è un singolo letterale positivo.

Es.: $(A \vee \neg B \vee \neg C) \equiv ((B \wedge C) \Rightarrow A)$

Nella forma di Horn, la premessa $((B \wedge C))$ si chiama **corpo** e la conclusione (A) **testa**.

Una formula costituita da un **singolo letterale positivo** (Es., A) viene chiamata **fatto**.
Anch'essa può essere scritta in forma di implicazione come $True \Rightarrow A$, ma è più semplice scrivere soltanto A .

- ❑ L'inferenza sulle clausole di Horn può essere svolta mediante gli algoritmi di concatenazione in avanti e concatenazione all'indietro (vedi prossime slide).
- ❑ Con le clausole di Horn è possibile determinare la conseguenza logica in un tempo che **cresce linearmente con la dimensione della base di conoscenza ($O(n)$)**



Concatenazione in Avanti (Forward chaining)

L'algoritmo di **concatenazione in avanti** **PL-CA-Consegue?(K, q)** determina se un singolo simbolo proposizionale **q** (la nostra query) è conseguenza logica della base di conoscenza **K** composta da **clausole definite**.

Descrizione intuitiva dell'algoritmo:

- L'algoritmo comincia dai **fatti noti** (clausole definite formate da un solo letterale positivo) presenti in K. Controlla poi se grazie a tali fatti è possibile verificare tutte le premesse di un'implicazione (**applicazione del Modus Ponent**), la sua conclusione è quindi aggiunta all'insieme dei **fatti noti**.
- Questo processo continua finché: (i) non viene aggiunta la stessa query **q** o (ii) non è più possibile effettuare alcuna inferenza.



Concatenazione in Avanti: algoritmo

function **PL-CA-Consegue?**(**K**, **q**) returns *true* oppure *false*

inputs: una base di conoscenza **K**, un simbolo proposizionale **q**

conto $\leftarrow$ una tabella, dove *conto*[*c*] è il numero di simboli della premessa della clausola *c*

inferiti $\leftarrow$ una tabella, dove *inferiti*[*s*] è inizialmente *false* per tutti i simboli

coda $\leftarrow$ una coda di simboli, che contiene inizialmente quelli noti come veri in **K** (i fatti noti)

while *coda* non è vuota **do**

p $\leftarrow$ Pop(*coda*)

if *p* = *q* **then return** *true*

if *inferiti*[*p*] = *false* **then**

inferiti[*p*] $\leftarrow$ *true*

for each clausola *c* in **K** con *p* in *c.Premessa* **do**

conto[*c*] $\leftarrow$ *conto*[*c*] - 1

if *conto*[*c*] = 0 **then** aggiungi *c.Conclusione* a *coda*

return *false*



Concatenazione in Avanti: esempio (1\8)

Consideriamo la seguente base di conoscenza **K**, la nostra query è **Q** (vogliamo quindi verificare se $K \models Q$)

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

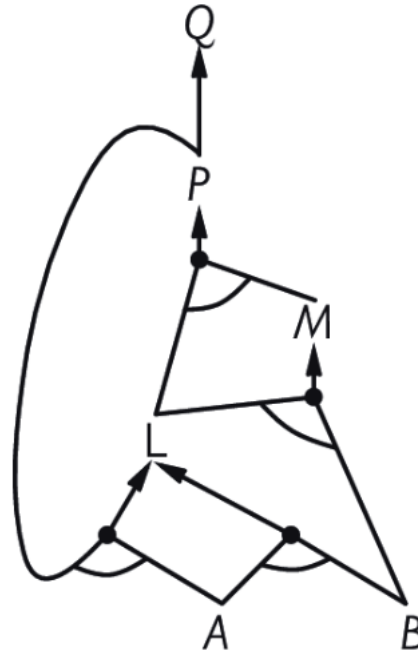
$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L$

A

B



Rappresentazione di **K** attraverso un grafo AND-OR. Più collegamenti uniti da un arco indicano una congiunzione

Concatenazione in Avanti: esempio (2\8)

Consideriamo la seguente base di conoscenza **K**, la nostra query è **Q** (vogliamo quindi verificare se $K \models Q$)

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

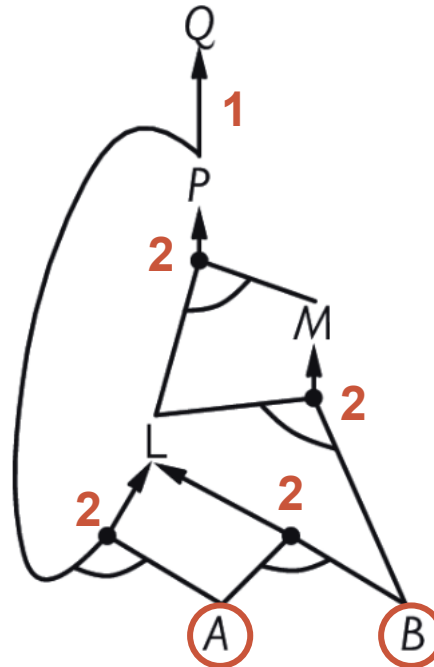
$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L$

A

B



L'algoritmo sceglie un fatto noto in **K** e verifica se si trova nella premessa di una congiunzione. Quando la premessa è soddisfatta aggiunge la conclusione (fatto) all'insieme delle formule.

Concatenazione in Avanti: esempio (3\8)

Consideriamo la seguente base di conoscenza **K**, la nostra query è **Q** (vogliamo quindi verificare se $K \models Q$)

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

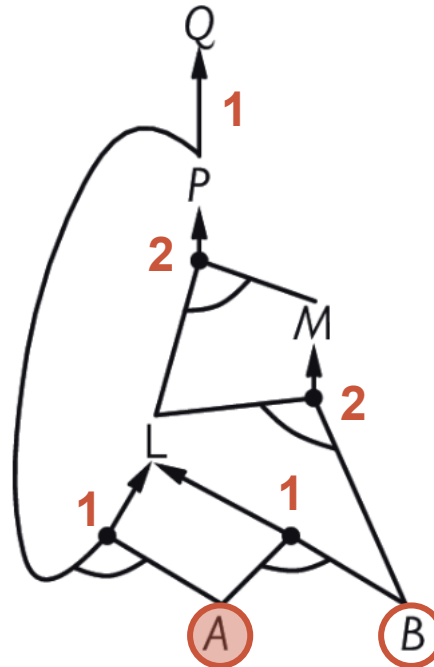
$B \wedge L \Rightarrow M$

A $\wedge P \Rightarrow L$

A $\wedge B \Rightarrow L$

A

B



L'algoritmo sceglie un fatto noto in **K** e verifica se si trova nella premessa di una congiunzione. Quando la premessa è soddisfatta aggiunge la conclusione (fatto) all'insieme delle formule.

Concatenazione in Avanti: esempio (4\8)

Consideriamo la seguente base di conoscenza **K**, la nostra query è **Q** (vogliamo quindi verificare se $K \models Q$)

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

B $\wedge$ L $\Rightarrow$ M

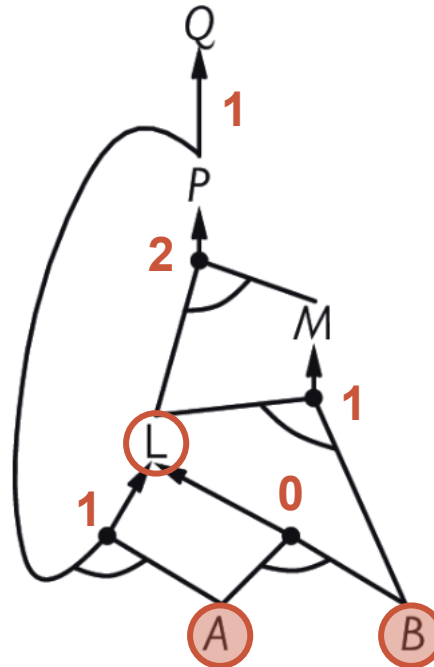
A $\wedge$ P $\Rightarrow$ L

A $\wedge$ **B** $\Rightarrow$ L

A

B

L



L'algoritmo sceglie un fatto noto in **K** e verifica se si trova nella premessa di una congiunzione. Quando la premessa è soddisfatta aggiunge la conclusione (fatto) all'insieme delle formule.

Concatenazione in Avanti: esempio (5\8)

Consideriamo la seguente base di conoscenza **K**, la nostra query è **Q** (vogliamo quindi verificare se $K \models Q$)

$P \Rightarrow Q$

L $\wedge$ M $\Rightarrow$ P

B $\wedge$ **L** $\Rightarrow$ M

A $\wedge$ P $\Rightarrow$ L

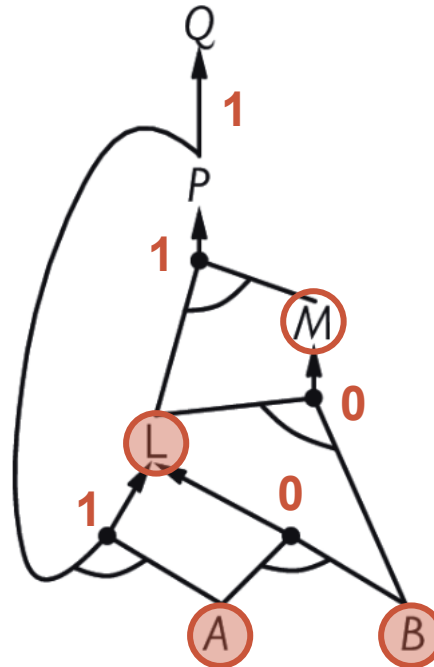
A $\wedge$ **B** $\Rightarrow$ L

A

B

L

M



L'algoritmo sceglie un fatto noto in **K** e verifica se si trova nella premessa di una congiunzione. Quando la premessa è soddisfatta aggiunge la conclusione (fatto) all'insieme delle formule.

Concatenazione in Avanti: esempio (6\8)

Consideriamo la seguente base di conoscenza **K**, la nostra query è **Q** (vogliamo quindi verificare se $K \models Q$)

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L$

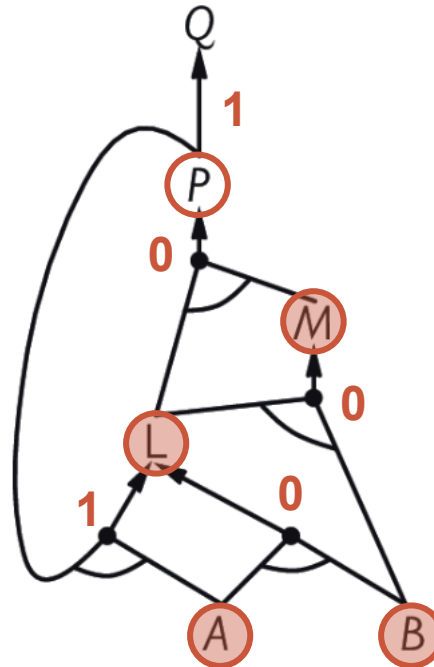
A

B

L

M

P



L'algoritmo sceglie un fatto noto in **K** e verifica se si trova nella premessa di una congiunzione. Quando la premessa è soddisfatta aggiunge la conclusione (fatto) all'insieme delle formule.

Concatenazione in Avanti: esempio (7\8)

Consideriamo la seguente base di conoscenza **K**, la nostra query è **Q** (vogliamo quindi verificare se $K \models Q$)

P $\Rightarrow$ **Q**

L $\wedge$ **M** $\Rightarrow$ **P**

B $\wedge$ **L** $\Rightarrow$ **M**

A $\wedge$ **P** $\Rightarrow$ **L**

A $\wedge$ **B** $\Rightarrow$ **L**

A

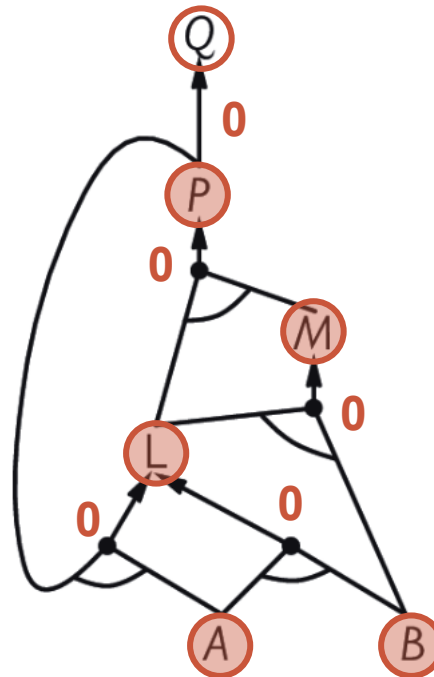
B

L

M

P

Q



L'algoritmo sceglie un fatto noto in **K** e verifica se si trova nella premessa di una congiunzione. Quando la premessa è soddisfatta aggiunge la conclusione (fatto) all'insieme delle formule.

Concatenazione in Avanti: esempio (8\8)

Consideriamo la seguente base di conoscenza **K**, la nostra query è **Q** (vogliamo quindi verificare se $K \models Q$)

P $\Rightarrow$ **Q**

L $\wedge$ **M** $\Rightarrow$ **P**

B $\wedge$ **L** $\Rightarrow$ **M**

A $\wedge$ **P** $\Rightarrow$ **L**

A $\wedge$ **B** $\Rightarrow$ **L**

A

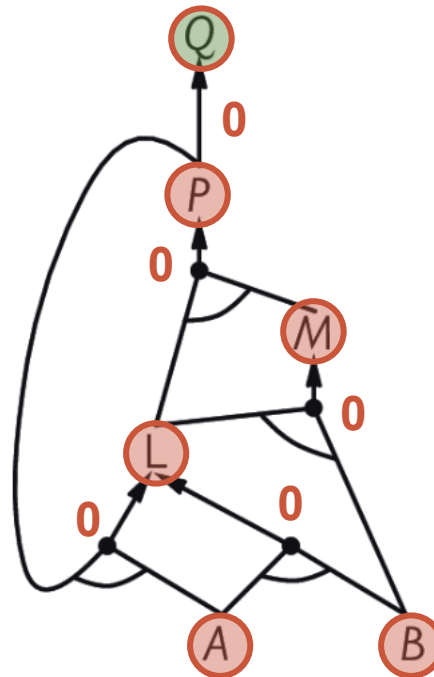
B

L

M

P

Q $\rightarrow$ return True



L'algoritmo sceglie un fatto noto in **K** e verifica se si trova nella premessa di una congiunzione. Quando la premessa è soddisfatta aggiunge la conclusione (fatto) all'insieme delle formule.

Concatenazione all'Indietro (Backward chaining)

L'algoritmo di **concatenazione all'indietro**, come suggerisce il nome, parte dalla query **q** e lavora a ritroso.

Se è già noto che la query è vera, non occorre fare nulla. In caso contrario, l'algoritmo trova tutte le implicazioni nella base di conoscenza che hanno **q** come conclusione: se tutte le premesse di una di quelle implicazioni possono essere dimostrate vere mediante la concatenazione all'indietro, allora **q** è vera.

Come nel caso della concatenazione in avanti, un'implementazione efficiente si esegue in **tempo lineare**.



Concatenazione in avanti VS Concatenazione all'indietro

La **concatenazione in avanti** è un esempio del concetto generale di **ragionamento guidato dai dati**, un tipo di ragionamento in cui l'attenzione parte dai fatti conosciuti.

All'interno di un agente quest'approccio può essere usato per derivare conclusioni partendo dalle percezioni in input, spesso anche senza avere in mente un quesito specifico.

Usato per task quali object recognition e routine recognition

La **concatenazione all'indietro** è una forma di ragionamento **basato sugli obiettivi**.

È utile per rispondere a questioni specifiche come “Cosa devo fare adesso?” oppure “Dove avrò lasciato le chiavi?”. Quindi più appropriato per il problem solving.

Spesso il costo della concatenazione all'indietro è molto **meno che lineare** nelle dimensioni della base di conoscenza, perché il processo **coinvolge solo i fatti rilevanti**.

